

Library Management System

Backend Architecture & API Reference

For Frontend Integration Team

1. System Architecture Overview

The LMS backend is a Spring Boot-based microservice system. It uses Spring Cloud for service discovery and centralised configuration. Three separate Spring Boot applications work in tandem.

Service	Port	Role	Notes
LMS Service	8081	Core API	Books, Members, Transactions, Reservations
Notification Service	8085	Email dispatch	SendGrid integration; internal only — not called by frontend
Eureka Server	8761	Service registry	Service discovery — not called by frontend
Config Server	8888	Config source	Pulls config from GitHub; not called by frontend

Frontend should only communicate with the LMS Service on port 8081. All other services are internal.

1.1 Base URL

Development	http://localhost:8081
All LMS routes prefixed with	/lms
Full example	http://localhost:8081/lms/getAllBooks

1.2 Common Response Format

Most endpoints return plain text strings (not JSON). Where JSON is returned it is documented per endpoint. All errors return a JSON ErrorResponse:

Content-Type (success)	text/plain or application/json
Content-Type (error)	application/json

HTTP error body schema	{ "message": "...", "status": 500, "timestamp": "2025-01-01T00:00:00" }
------------------------	---

1.3 Rate Limiting

All /lms/** routes are rate-limited. The interceptor runs before every request.

Dev limit	100 requests per minute per IP
Prod limit	20 requests per minute per IP
Over-limit HTTP status	429 Too Many Requests
Over-limit body	{ "error": "Rate limit exceeded. Try again later." }

1.4 Circuit Breaker (Resilience4j)

BorrowBook and ProcessReturn are wrapped with a circuit breaker. When the circuit is OPEN, fallback messages are returned with HTTP 200.

Circuit name	bookService
Failure threshold (dev)	80%
Failure threshold (prod)	50%
OPEN state wait (dev)	5 seconds
OPEN state wait (prod)	30 seconds
Fallback (borrow)	"The borrowing service is temporarily unavailable..."
Fallback (return)	"The return service is temporarily unavailable..."
Monitor endpoint	GET /lms/circuitBreakers

2. Core Data Models

2.1 Member

Field	Type	Nullable	Notes
id	Long	No	Auto-generated random ID
userName	String	No	Unique login name
password	String	No	Stored in plain text — hash before displaying
emailId	String	No	Used for notifications

phoneNumber	String	Yes	Optional contact
membershipStartDate	LocalDate	No	Set at registration
membershipEndDate	LocalDate	No	start + memberShipMonths
membershipPlan	IMembershipPlan	No	STANDARD or PREMIUM (+ optional add-ons)
transactions	List<Transaction>	No	All borrow records
reservations	List<Reservation>	No	All reservations
notifications	List<Notification>	No	Sent notification log

2.2 Book

Field	Type	Nullable	Notes
id	Long	No	Auto-generated random ID
isbn	String	Yes	International book number
title	String	No	Book title
author	String	No	Author name
category	String	Yes	Genre / category
publicationYear	Integer	Yes	Year of publication
copiesAvailable	Integer	No	Count of available copies; 0 = Unavailable
currentState	State	No	Available Unavailable Lost
reservations	List<Reservation>	No	
transactions	List<Transaction>	No	

2.3 Transaction

Field	Type	Nullable	Notes
id	Long	No	Auto-generated
borrowDate	LocalDateTime	No	Set on creation
returnedDate	LocalDateTime	Yes	Set on return
transactionStatus	Enum	No	ACTIVE RETURNED FINE_PENDING
member	Member	No	Borrower reference
book	Book	No	Borrowed book reference

2.4 Reservation

Field	Type	Nullable	Notes
id	Long	No	Auto-generated
member	Member	No	Reserving member
book	Book	No	Reserved book
reservationDate	LocalDate	No	Date created
expiryDate	LocalDate	Yes	Set when status → ACTIVE; null while QUEUED
status	Enum	No	QUEUED ACTIVE FULFILLED EXPIRED

2.5 Membership Plans

Attribute	Standard Plan	Premium Plan	+ Extended Borrowing Add-on
PlanType enum value	STANDARD	PREMIUM	Applied on top of either plan
Cost (\$/month)	\$5	\$7	+\$3
Max Reservations	2	5	No change
Borrowing Limit	10 books	10 books	+2 books
Borrow Duration	30 days	30 days	+7 days (37 days)
Fine Rate	\$0.25/day	\$0.25/day	No change

⚠ The EXTENDED_BORROWING add-on is the only available add-on currently. Pass it in the addOns field of RegisterMemberRequest.

3. Enumerations

Enum	Values	Usage
PlanType	STANDARD, PREMIUM	RegisterMemberRequest.planType
AddOns	EXTENDED_BORROWING	RegisterMemberRequest.addOns (optional)
TransactionStatus	ACTIVE, RETURNED, FINE_PENDING	Transaction state machine
ReservationStatus	QUEUED, ACTIVE, FULFILLED, EXPIRED	Reservation lifecycle
NotificationType	BORROWED, RETURNED, RESERVED, OVERDUE, RESERVATION_CREATED, RESERVATION_AVAILABLE	Notification event type — internal use

4. API Endpoints

All endpoints are under `http://localhost:8081/lms/` in development. The table below summarises all available routes. Detailed docs follow.

Method	Path	Domain	Summary
POST	/lms/registerMember	Members	Register a new library member
POST	/lms/addBook	Books	Add a new book to inventory
GET	/lms/getAllBooks	Books	List all books
POST	/lms/removeBook/{bookId}	Books	Mark a book as unavailable (lost)
POST	/lms/undoLastBookOperation	Books	Undo the last add/remove operation
GET	/lms/commandHistory	Books	View command history
DELETE	/lms/commandHistory	Books	Clear command history
GET	/lms/borrowBook/	Transactions	Borrow a book
GET	/lms/processReturn/{transactionId}	Transactions	Return a borrowed book
POST	/lms/reserveBook	Reservations	Reserve an unavailable book
POST	/lms/processReservationPickup/{id}	Reservations	Confirm reservation pickup
POST	/lms/expireActiveReservations	Reservations	Expire overdue reservations (scheduler)
GET	/lms/circuitBreakers	Monitoring	View circuit breaker status

4.1 Member Endpoints

POST /lms/registerMember	
Description	Register a new library member with a selected membership plan.
Auth Required	None
Request Body	<pre>{ "userName": "john_doe", // required "password": "secret123", // required "emailId": "john@example.com", // required - used for notifications "phoneNumber": "0871234567", // optional "planType": "STANDARD", // required - STANDARD PREMIUM "memberShipMonths": 12, // required - integer (months of membership) "addOns": "EXTENDED_BORROWING" // optional - omit if not needed }</pre>

200 Success	Member created with Id : 1234567890123456 Selected Plan of Member Standard Plan Total Cost of the Plan 5
500 Error	Missing required fields — message describes which field
429 Error	Rate limit exceeded
Notes	The returned ID is required for borrowing and reserving books. Store it in the frontend session.

4.2 Book Endpoints

POST /lms/addBook	
Description	Add a new book to the library inventory. Uses the Command pattern — can be undone.
Auth Required	None
Request Body	<pre>{ "isbn": "978-0132350884", // optional "title": "Clean Code", // required "author": "Robert Martin", // required "category": "Technology", // optional "publicationYear": 2008, // optional "copiesAvailable": 5 // required — must be >= 1 }</pre>
200 Success	Book added with Id: 9876543210987654
500 Error	Validation errors on required fields
429 Error	Rate limit exceeded
Notes	Save the returned book ID for subsequent borrow/reserve operations.

GET /lms/getAllBooks	
Description	Retrieve all books in the system including their current state and copy count.
Auth Required	None
200 Success	<pre>[{ "id": 9876543210987654, "isbn": "978-0132350884", "title": "Clean Code", "author": "Robert Martin", "category": "Technology", "publicationYear": 2008, "copiesAvailable": 5, "currentState": { "stateName": "Available" } }]</pre>
Notes	The membershipPlan field is excluded (@JsonIgnore). Reservation/transaction lists are included but can be large.

POST /lms/removeBook/{bookId}	
Description	Mark a book as Lost and set copiesAvailable to 0. This is soft-delete — it can be undone.
Auth Required	None
Path Params	bookId — Long — the book's numeric ID
200 Success	Book marked as unavailable: Clean Code
500 Error	{ "message": "Book not found with ID: 123", "status": 500, ... }

Notes	This operation is stored in command history. Call <code>/lms/undoLastBookOperation</code> to reverse.
--------------	---

POST <code>/lms/undoLastBookOperation</code>	
Description	Undo the most recent addBook or removeBook operation.
Auth Required	None
200 Success	Book addition undone: ID 987 marked as unavailable // OR Book 'Clean Code' restored with 5 copies available
200 Error	"Cannot undo: No commands to undo" (returns 200 with message, not an error status)
Notes	History is capped at 50 entries. Undo is destructive — it pops the command from history.

GET <code>/lms/commandHistory</code>	
Description	View the current command history stack.
Auth Required	None
200 Success	<pre>{ "historySize": 3, "canUndo": true, "lastCommand": "Add Book: 'Clean Code' by Robert Martin (ISBN: 978-0132350884)", "allCommands": ["Add Book: ...", "Remove Book: ...", "Add Book: ..."] } // OR when empty: { "message": "No commands in history", "historySize": 0, "canUndo": false }</pre>

DELETE <code>/lms/commandHistory</code>	
Description	Clear all entries from the command history.
Auth Required	None
200 Success	Command history cleared successfully

4.3 Transaction Endpoints

⚠ Note: Both transaction endpoints use GET HTTP method. This is a design choice in the codebase — treat them as actions despite the method.

GET <code>/lms/borrowBook/</code>	
Description	Borrow a book for a member. Decrements available copies. Triggers email notification.
Auth Required	None
Query Params	book_id (Long, required) — ID of the book to borrow member_id (Long, required) — ID of the borrowing member Example: <code>/lms/borrowBook/?book_id=123&member_id=456</code>
200 Success	Book borrowed successfully. Transaction ID: 8765432109876543 Please note this ID — it is required when returning the book.

500 Error	"Book 'Clean Code' is not available. Please reserve the book..."
500 Error	"Borrowing limit exceeded. Current: 10, Limit: 10"
500 Error	"Member not found with ID: 456"
500 Error	"Book not found with ID: 123"
200 Error	Circuit breaker OPEN — returns fallback message with HTTP 200
429 Error	Rate limit exceeded
Notes	Store the Transaction ID — it is mandatory for processReturn. The book must be Available and the member must not have exceeded their plan's borrowing limit.

GET /lms/processReturn/{transactionId}	
Description	Return a borrowed book. If overdue, calculates and returns fine details. Notifies member. Activates next queued reservation if any exists.
Auth Required	None
Path Params	transactionId — Long — the transaction ID returned from borrowBook
200 Success	// On-time return: Book 'Clean Code' returned successfully ON TIME. Thank you for returning by the due date (2025-05-01T10:00)! // Overdue return: Book 'Clean Code' returned LATE. Overdue fine: \$3.50 (14.0 days late). Due date was: 2025-04-01T10:00. Returned on: 2025-04-15T09:00
500 Error	"Transaction not found with ID: 123"
500 Error	"Transaction #123 is already closed. Status: RETURNED"
200 Error	Circuit breaker OPEN — returns fallback message with HTTP 200
429 Error	Rate limit exceeded
Notes	After return, if the book has queued reservations, the next in queue is automatically activated and the reserving member is notified.

4.4 Reservation Endpoints

POST /lms/reserveBook	
Description	Reserve a book that is currently unavailable. Adds to reservation queue. Member is notified with their queue position.
Auth Required	None
Query Params	book_id (Long, required) — ID of the book to reserve member_id (Long, required) — ID of the member Example: /lms/reserveBook?book_id=123&member_id=456
200 Success	Reservation created with id:5432109876543210 Your reservation queue number is:1
500 Error	"Book is currently AVAILABLE — borrow it directly instead of reserving"
500 Error	"You already have an active reservation for this book"

500 Error	"Reservation limit exceeded. Current: 2, Limit: 2"
500 Error	"Member not found" or "Book not found"
429 Error	Rate limit exceeded
Notes	A member can only reserve a book if it is Unavailable. If Available, direct them to borrowBook. The reservation starts in QUEUED status.

POST /lms/processReservationPickup/{reservationId}	
Description	Confirm a member is picking up their reserved book. Transitions reservation from ACTIVE → FULFILLED.
Auth Required	None
Path Params	reservationId — Long — the reservation ID returned from reserveBook
200 Success	Reservation fulfilled. Book borrowing successfull.
500 Error	"Reservation not found with ID: 123"
500 Error	"Can only fulfill ACTIVE reservations. Current status: QUEUED"
500 Error	"Book is not available. Status: Unavailable"
429 Error	Rate limit exceeded
Notes	The reservation must be in ACTIVE status (not QUEUED). A reservation becomes ACTIVE when the book is returned and the member is at the front of the queue.

POST /lms/expireActiveReservations	
Description	Scan all ACTIVE reservations and expire those past their expiry date. Intended for scheduled/cron invocation — can also be triggered manually from admin UI.
Auth Required	None
200 Success	(empty body — HTTP 200)
Notes	Active reservations have a 2-day pickup window. After expiry, the slot opens for the next person in the queue.

4.5 Monitoring Endpoint

GET /lms/circuitBreakers	
Description	Returns real-time state of all Resilience4j circuit breakers. Useful for admin dashboards.
Auth Required	None
200 Success	<pre>{ "bookService": { "state": "CLOSED", "failureRate": "-1.0%", "numberOfBufferedCalls": 5, "numberOfFailedCalls": 0, "numberOfSuccessfulCalls": 5, "description": "Normal operation - requests passing through" } } // Possible states: CLOSED (normal), OPEN (failing),</pre>

	HALF_OPEN (recovering) // When no circuit breakers have been invoked yet: { "message": "No circuit breakers have been invoked yet. Call /lms/borrowBook/ or..." }
Notes	Circuit breaker state transitions automatically. CLOSED = healthy, OPEN = fallback active, HALF_OPEN = testing recovery.

5. End-to-End Frontend Workflows

5.1 Register Member and Borrow a Book

#	Action	Expected outcome / what to store
1	POST /lms/registerMember	Parse the returned ID from the response string. Store as memberId.
2	POST /lms/addBook	Parse the returned ID from the response string. Store as bookId.
3	GET /lms/getAllBooks	Confirm book appears and copiesAvailable > 0.
4	GET /lms/borrowBook/?book_id=X&member_id=Y	Parse Transaction ID from the response string. Store as transactionId.
5	GET /lms/processReturn/{transactionId}	Check for LATE vs ON TIME message. Extract fine if present.

5.2 Reserve and Pick Up a Book

#	Action	Expected outcome / what to store
1	POST /lms/reserveBook?book_id=X&member_id=Y	Parse Reservation ID from response string. Store as reservationId. Note queue position.
2	(Automatic) Another member returns the book	LMS activates first queued reservation. Member receives email notification.
3	POST /lms/processReservationPickup/{reservationId}	Expect: "Reservation fulfilled. Book borrowing successful." Reservation moves to FULFILLED.

⚠ Reservations can only be made on UNAVAILABLE books. If a user tries to reserve an available book, return a UI-level error before even calling the API.

6. Parsing Plain-Text Responses

Most LMS endpoints return plain text. Below are regex/split patterns to extract IDs from responses.

Endpoint	Example response fragment	Parse pattern
registerMember	Member created with Id : 123456789	response.split("Id : ")[1].split("\n")[0]
addBook	Book added with Id: 987654321	response.split("Id: ")[1]
borrowBook	Transaction ID: 111222333	response.split("Transaction ID: ")[1].split("\n")[0]
reserveBook	Reservation created with id:555666777	response.split("id:")[1].split("\n")[0]
processReturn (fine)	Overdue fine: \$3.50	response.match(/fine: \\$([d.]+)/)[1]

7. Error Handling

All unhandled exceptions are caught by the GlobalExceptionHandler and returned as HTTP 500 with this JSON body:

Schema	{ "message": "...", "status": 500, "timestamp": "2025-..." }
Content-Type	application/json
Status Code	500 Internal Server Error (for all business logic errors)

HTTP Status	Condition	Frontend recommendation
200 (fallback)	Circuit breaker OPEN — response body contains fallback message	Show service unavailable banner. Retry after delay.
429	Rate limit exceeded — too many requests per IP	Show cooldown timer. Disable submit buttons temporarily.
500	Business logic error (book not found, limit exceeded, etc.)	Parse message field from JSON body and display to user.

8. Key Design Patterns (for Frontend Context)

Pattern	Where used	Frontend impact
Strategy	IMembershipPlan — StandardPlan, PremiumPlan	No direct impact. Plan type is selected at registration.
Decorator	ExtendedBorrowingDecorator wraps a plan	Pass addOns field during registration to apply.
Factory	MembershipPlanFactory resolves plan from PlanType	No direct impact — backend handles plan resolution.

Command + Undo	AddBookCommand, RemoveBookCommand	Expose Undo button in admin UI using /lms/undoLastBookOperation.
State	Book states: Available, Unavailable, Lost	Check currentState.stateName in book objects before allowing borrow/reserve.
Circuit Breaker	borrowBook, processReturn	Handle 200 fallback responses — check for expected success message vs fallback string.
Decorator (AOP)	LoggerAdvice logs all service calls	No frontend impact — purely backend logging.
Interceptor	RateLimitInterceptor, RequestLoggingInterceptor	Handle 429 responses gracefully.